# Agentic Threat Hunting On The ELK Stack

SE6014 – Security Monitoring and Threat Detection

## Project Report

Group 3


**Submitted By**

Pratyush Joshi – G2610291K
Madhur Choudhury – G2607363G
Madhav Malhotra – G2607476J
Aleksander Sak – G2610054E
Naren Rithvik – G2602781B
Victor Moll Pons – G2609992L


In partial fulfilment for the award of the degree of

**MASTER OF SCIENCE**

in

**CYBERSECURITY**


Nanyang Technological University, Singapore – 639798

October, 2026

# Contents

**A Executive Summary** **2**

**1 System Architecture** **3**

1.1 Components of the System . . . . . 3

1.2 Data Flow Between Components . . . . . 6

1.3 Tools Available to the Agents . . . . . 6

1.4 Decision-Making Logic . . . . . 7

**2 Setup Documentation** **7**

2.1 Installation and Configuration . . . . . 7

2.2 Challenges Faced During Development . . . . . 10

2.3 System Requirements and Environment Setup . . . . . 11

**3 Agentic Workflow Design** **12**

3.1 Completeness and Accuracy . . . . . 12

3.2 Guardrails and Safety . . . . . 13

**4 Experiment Results** **14**

4.1 Comparison Run (Network Analysis) . . . . . 14

4.2 Comparison Run (Recorded Analysis) . . . . . 16

4.3 Comparison Run (Ubuntu Analysis) . . . . . 20

**5 Conclusion** **23**

5.1 Limitations of the Current System . . . . . 23

5.2 Future Work and Enhancements . . . . . 23

**6 Appendix A** **24**

6.1 Timesheet Breakdown & Responsibility . . . . . 24

# A Executive Summary

With today's technology, when a security team has a hunch about an attack, an analyst would have to manually go through thousands of computer logs, develop a specific query on his own, and build an evidence trail from scratch, a procedure that can take hours or even days. With our software, all of this is done automatically. You provide your suspicion in a layman way, such as questioning if there is somebody trying to access our servers at nighttime, and the software goes through the appropriate logs, finds the proof, and delivers you its decision backed up with evidence in a matter of minutes. You get your security team an automated junior analyst who works non-stop and can search through millions of logs in seconds without having to worry about any breaches.

Our tests were conducted using actual security log data with specific threat scenarios given to the system to examine. The system was able to detect abnormal activity, collect evidence, and present its findings in form of comprehensive reports completely independently from human interference halfway through the process. We repeated the same hunting hypothesis experiments several times in all three logging environments to test for any consistency. While the results were not exactly the same every time, the second run results showed significantly better accuracy, wider threat range, and lesser hallucination, which is likely a consequence of the model fine tuning done between runs. This suggests that reasoning by the system may improve with training, and the results, while not yet fully consistent, show a trend towards accuracy.

The system cannot yet run in a fully autonomous, continuous mode due to the computational demands of the AI agent. Manual prompting and occasional restarts remain part of the workflow when processing large log volumes.

# 1 System Architecture

## 1.1 Components of the System

### 1.1.1 ELK Stack (Elasticsearch, Logstash, Kibana)

- **Logstash** – receives incoming logs via pull, push, and stream; parses and normalizes them; applies schema templates
- **Elasticsearch** – stores all parsed logs in a searchable index; the database your agents query against
- **Kibana** – visual dashboard layer; shows monitoring statistics, events processed, detections raised

### 1.1.2 Tailscale

- A VPN mesh network that securely connects all the VMs together
- Allows the Windows VM, Ubuntu Desktop, and Ubuntu Server to talk to each other privately without exposing anything to the outside

### 1.1.3 VMware

- The virtualization platform that hosts all the VMs
- Everything runs inside VMware, making it self-contained and safe to test on
- Allows us to snapshot machines before running attacks, giving us a rollback point

### 1.1.4 Ubuntu Desktop

- The main analyst workstation
- This is where we orchestrate, view the dashboards and interact with the system itself

### 1.1.5 Windows VM

- The target endpoint being monitored and defended
- Where Atomic Red Team attack simulations are executed
- Generates the Windows event logs and Sysmon logs that feed into the pipeline

### 1.1.6 Ubuntu Server

- The backend server running ELK, Ollama, and any other services
- The workhorse of the system most of the heavy processing happens here

### 1.1.7 Ubuntu Server Firewall

- Ubuntu Server firewall logs generated by the Uncomplicated Firewall (UFW) service
- Captures allow, deny, and audit events for network traffic passing through the Ubuntu Server
- Serves as the third distinct log type alongside Windows Event Logs and Sysmon, satisfying the requirement for at least three log sources

### 1.1.8 Atomic Red Team

- Not a log source itself, but the attack simulation framework
- Runs benign test scripts on the Windows VM that mimic real attacker techniques (following MITRE ATT&CK)
- Generates the suspicious activity that your system needs to detect
- Used to produce the evidence in the experiment runs

### 1.1.9 OMLX (Open MLX)

- Runs the AI models locally on Apple Silicon Macs, providing the inference runtime for Gemma 4 and Qwen 3.5
- Enables local model execution without requiring cloud-based inference, ensuring no log data or query content leaves the private network
- Works alongside the log shipping pipeline to keep all processing contained within the Tailscale network

### 1.1.10 Gemma 4

- One of our locally running LLMs
- It is used for a specific reasoning task e.g. schema generation, hypothesis planning, or report writing
- Being local means no data leaves your environment, which is important for a security product

### 1.1.11 Qwen 3.5

- The second locally-running LLM
- Could be used for a different task from Gemma e.g. triage decisions, evidence summarization, or log classification

### 1.1.12 Filebeat

- Collects and ships system and audit logs from the Ubuntu Server to Elasticsearch
- Enables UFW firewall logs and auditd logs to be ingested into the pipeline via the system and auditd modules

#### 1.1.13 Winlogbeat

- Collects Windows Event Logs from the Windows VM and forwards them to Elasticsearch as one of the three distinct log sources
- Configured to capture Windows Security, System, PowerShell, and Sysmon event channels

#### 1.1.14 Sysmon64

- Generates detailed Sysmon logs as a second distinct log source, providing deeper endpoint visibility than native Windows Event Logs alone
- Captures process creation, network connections, and LSASS access events that were directly used in the Mimikatz experiment runs

#### 1.1.15 MCP ELK Server

- Acts as the bridge between the AI Hunt Agent and Elasticsearch, exposing search and aggregation capabilities as callable tools
- Ensures the agent interacts with log data through a controlled, read-only interface rather than directly accessing Elasticsearch

#### 1.1.16 AI Hunt Agent

- The primary orchestrator of the threat hunt, receiving hypotheses in plain language and planning, executing, and interpreting queries against the log data via the MCP server
- Powered by Gemma 4 as the primary reasoning model, with Qwen 3.5 available as backup

#### 1.1.17 Parsing Agent

- Processes and structures raw log data before it reaches Elasticsearch, normalising fields and enriching records for consistent querying
- Powered by Qwen 3.5, keeping the parsing workload separate from the reasoning workload handled by Gemma 4

#### 1.1.18 Unsloth

- A lightweight fine-tuning framework used to train the Gemma 4 and Qwen 3.5 models on security specific log patterns
- Enabled targeted model training between output runs to reduce hallucination in the agent's reasoning

#### 1.1.19 PCAP / Network Logs

- Packet capture logs representing raw network traffic between machines in the Tailscale private network, capturing source IP, destination IP, port, and protocol at the packet level
- Complements UFW logs by capturing the full content of network communications rather than just firewall decisions

**Physical machine layout**

| Physical Machine | Hardware | What Runs On It |
|---|---|---|
| PC 1 – Hunt Machine | Apple Silicon Mac | AI Hunt Agent (Gemma 4 via OMLX, with Qwen 3.5 as backup), MCP ELK Server, Unsloth fine-tuning |
| PC 2 – Ubuntu Desktop / ELK Host | Physical machine | ELK Stack: Elasticsearch, Kibana, Logstash |
| PC 3 – Vulnerable Machines | Physical machine | VMware VMs running Windows VM and Ubuntu Server |
| Separate logical component | Runs on Hunt Machine | Parsing Agent (Qwen 3.5) |

## 1.2 Data Flow Between Components

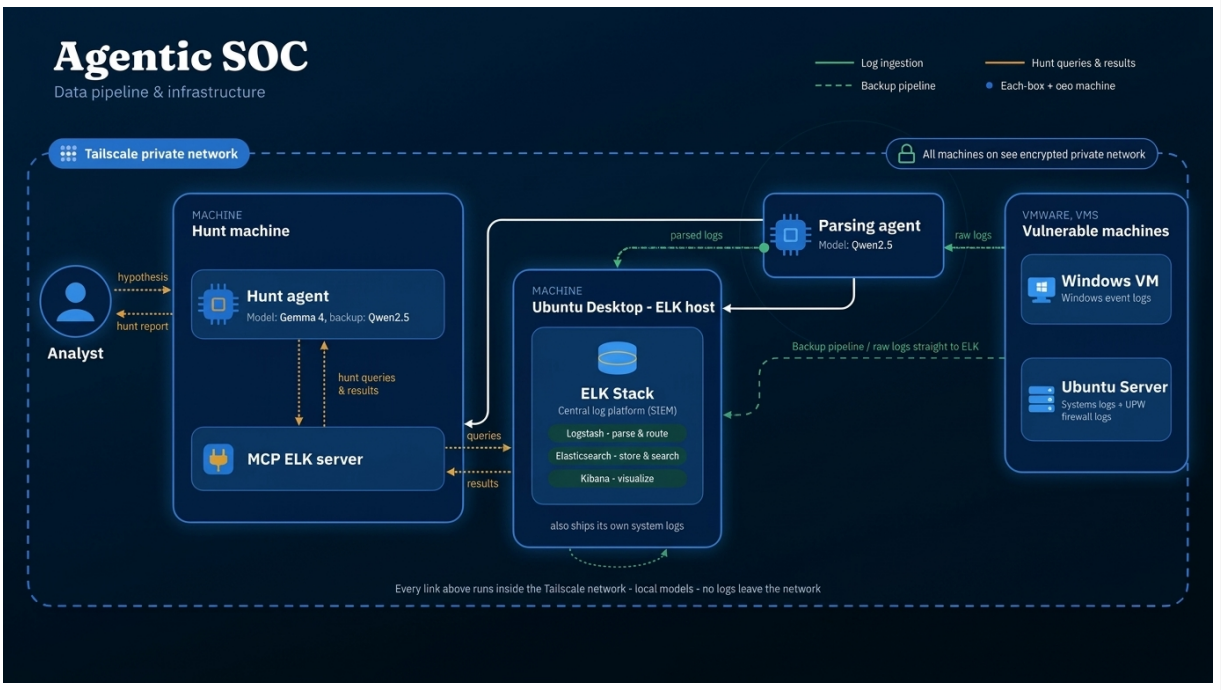



Figure 1: Data Pipeline Data Flow

## 1.3 Tools Available to the Agents

- **MCP Server:** The Model Context Protocol (MCP) server acts as the bridge between the AI Hunt Agent and the security data stored in Elasticsearch. It exposes a set of tools that the agent can call to retrieve logs, search for suspicious activity, and gather evidence during an investigation. The available tools and their permissions determine what information the agent can access and what operations it can perform.
- **Elasticsearch:** Provides the agent with access to indexed security logs through MCP tools. Depending on the tools implemented, the agent can search Windows and Linux events, filter records by time range or event type, and retrieve relevant evidence for investigating potential threats.

- **Gemma 4:** Serves as the primary reasoning model for the AI Hunt Agent, if configured in this role. It can interpret an investigation hypothesis, examine retrieved evidence, identify relationships between events, and summarise suspicious activity into understandable findings.
- **Qwen 3.5:** Can support classification and triage tasks if assigned this role in the implementation. For example, it can categorise events by potential severity, help prioritise suspicious records, and structure findings for further investigation. Its exact responsibilities depend on how the models are configured.
- **Kibana:** Provides the visual monitoring interface for analysts to inspect collected logs, explore dashboards, and verify investigation findings. It is primarily used by the human analyst rather than directly by the AI agent, unless specific Kibana tools or integrations have been implemented.

### 1.4 Decision-Making Logic

- **Gemma 4 – Hunt Planning:** Acts as the primary reasoning model, interpreting the analyst's hypothesis and planning which security logs and events need to be investigated to find evidence of suspicious activity.
- **Qwen 3.5 – Event Classification and Triage:** Helps categorise collected events, assess their potential severity, and prioritise suspicious activity for further investigation.
- **MCP Server – Tool Selection and Execution:** Provides the AI Hunt Agent with tools to query Elasticsearch and retrieve relevant logs. The agent selects tools based on the investigation requirements, using the returned evidence to guide subsequent queries.
- **Atomic Red Team – Detection Validation:** Supplies controlled attack simulations that represent behaviours the system should detect. The resulting logs can be used to evaluate whether the agents identify and correctly investigate suspicious activity.

## 2 Setup Documentation

### 2.1 Installation and Configuration

#### Step 1: Environment Setup

Provision the three physical machines and the two virtual machines as follows:

- **PC 1 – Hunt Machine (Apple Silicon Mac):** Hosts the AI Hunt Agent, MCP ELK Server and Parsing Agent.
- **PC 2 – Ubuntu Desktop / ELK Host:** Hosts Elasticsearch, Kibana and Logstash.
- **PC 3 – VMware Host:** Runs two virtual machines using VMware:
  - **Ubuntu Server:** Generates Linux system and authentication logs.
  - **Windows VM:** Runs Sysmon, Winlogbeat and Atomic Red Team for controlled attack simulations.

- Ensure each physical machine meets the minimum specifications outlined in Section 2.3 before provisioning VMs.
- On the Hunt Machine (Apple Silicon Mac), verify that OMLX is compatible with the hardware architecture before proceeding all model inference runs on Apple Silicon via Metal acceleration.

- Take a VMware snapshot of the Windows VM and Ubuntu Server before any Atomic Red Team simulation is run, providing a clean rollback point between experiment runs.
- Assign static Tailscale IP addresses to each machine at this stage to avoid connectivity issues during later configuration steps.

### Step 2: Tailscale Network Setup

Install Tailscale on all three machines and sign in to the same private network. Verify connectivity and restrict access to authorized lab devices.

- After installation, verify that all three physical machines and the VMs are visible to each other using Tailscale's admin console before proceeding.
- Configure Tailscale ACLs to restrict which machines can communicate with which — the Windows VM should only be able to reach the ELK host, not the Hunt Machine directly.
- Disable direct local network access between machines where possible, routing all inter-machine traffic through Tailscale exclusively to ensure no logs travel over an unencrypted path.

### Step 3: ELK Stack Installation

On Ubuntu Desktop, install Elasticsearch and Kibana using Elastic's official Debian packages. Configure authentication, TLS certificates and private network access, then verify that Kibana can connect to Elasticsearch.

- Pin all Elastic stack components (Elasticsearch, Kibana, Logstash) to the same version before installation version mismatches between components are a common source of authentication and API compatibility failures.
- After configuring TLS, test the Elasticsearch endpoint directly using `curl` before connecting Kibana, to isolate any certificate issues early.
- Create a dedicated read-only API key in Elasticsearch specifically for the AI Hunt Agent — the agent should never use admin credentials to query logs.
- Create separate Elasticsearch indices for each log type (Windows events, Sysmon, UFW/firewall, Ubuntu system logs) rather than a single combined index, to allow the agent to target specific log sources during a hunt.
- Configure index lifecycle management (ILM) policies to prevent Elasticsearch from filling the disk during high-volume log ingestion periods.
- Official guide: Elastic Debian installation (*Install Elasticsearch with a Debian package* | Elastic Docs).

### Step 4: Linux Log Collection – Filebeat

Install Filebeat on Ubuntu Server using Elastic's APT repository. Enable the system module, configure the Elasticsearch endpoint and credentials, and start the service to forward Linux logs to Elasticsearch.

- Enable both the system module and the auditd module in Filebeat to capture UFW firewall logs alongside standard system logs UFW logs are not collected by the system module alone.
- After starting the service, verify logs are arriving in Elasticsearch by checking the Kibana

Discover view before proceeding a misconfigured Elasticsearch endpoint or incorrect credentials will cause Filebeat to fail silently.

- If the auditd parsing error (`field [auditd.log.kv] does not contain value_split`) appears in Kibana, this is a known Filebeat configuration issue with the audit log format and not a system threat adjust the Filebeat auditd module configuration to resolve it.
- Guide: Filebeat installation (*Filebeat quick start: installation and configuration* | Beats).

### Step 5: Windows Monitoring Sysmon64 and Winlogbeat

Install Sysmon64 on the Windows VM to capture system activity, including process execution and network events. Install Winlogbeat and configure it to forward Windows Security, System, PowerShell and Sysmon logs to Elasticsearch.

- Use a well-established Sysmon configuration file such as the SwiftOnSecurity config as a baseline, rather than the default configuration the default captures insufficient detail for threat hunting.
- Configure Winlogbeat to forward the following specific event channels: Windows Security, Windows System, Windows PowerShell, and the `Microsoft-Windows-Sysmon/Operational` channel missing any of these will create visibility gaps the agent cannot compensate for.
- After configuration, verify that Sysmon Event ID 10 (process access) and Event ID 1 (process creation) are appearing in Kibana before running any Atomic Red Team tests these are the primary events used in the LSASS hunt.
- Confirm that PowerShell Script Block Logging (Event ID 4104) is enabled via Group Policy on the Windows VM this is required to capture fileless execution techniques such as IEX commands.
- Guide: Microsoft Sysmon (*Sysmon Sysinternals*); Winlogbeat setup (*Winlogbeat quick start: installation and configuration* | Beats).

### Step 6: Attack Simulation – Atomic Red Team

Install Atomic Red Team and its PowerShell execution framework on the isolated Windows VM. Run selected, reviewed tests to simulate controlled adversary behaviour and verify that Sysmon and Winlogbeat capture the resulting events.

- Before running any tests, verify that Windows Defender and any other antivirus on the Windows VM is either disabled or configured to exclude the Atomic Red Team directory live antivirus will block many test executions before Sysmon can capture them.
- Run tests individually rather than in bulk to bulk execution generates log volumes that can overwhelm the ingestion pipeline and cause the delays observed during testing.
- After each test, allow at least 60 seconds before querying the agent to ensure all generated events have been ingested into Elasticsearch running the hunt immediately after the simulation may return incomplete results due to the ingestion delay identified in our testing.
- Verify that each test's expected events appear in Kibana before submitting the hunt hypothesis to the agent if the logs are not visible in Kibana, the agent cannot find them regardless of how the hypothesis is phrased.
- Guide: Atomic Red Team - GitHub: `https://github.com/redcanaryco/atomic-red-team` (*Small and highly portable detection tests based on MITRE's ATT&CK*).

### Step 7: Model Fine Tuning (Unsloth)

Install Unsloth on the Hunt Machine to enable local fine-tuning of the Gemma 4 and Qwen 3.5 models on Apple Silicon hardware.

- Configure Unsloth with a curated dataset of security log patterns, expected reasoning chains, and correct MITRE ATT&CK mappings drawn from the experiment log data.
- Run fine-tuning passes between output runs to reduce hallucination and improve the accuracy of process tree reconstruction, technique mapping, and IOC extraction.
- Validate the fine-tuned models by re-running the same hunt hypotheses and comparing output quality against the pre-training baseline.
- Guide: Unsloth documentation (`https://github.com/unslothai/unsloth`).

### Step 8: AI Hunt Agent and MCP Integration

Configure the AI Hunt Agent to communicate with Elasticsearch through the MCP ELK Server using a dedicated, read-only API key. The agent searches collected logs, correlates suspicious events and produces an investigation report containing timelines, evidence and recommendations.

- Create separate Kibana dashboards for each log source Windows events, Sysmon, Ubuntu system logs, and UFW firewall logs rather than a single combined view, to mirror how the agent queries each index separately.
- Configure time filters in Kibana to match the exact window used in each hunt hypothesis this allows direct comparison between what Kibana shows and what the agent reports.
- After each experiment run, export the relevant Kibana dashboard as a PDF or screenshot to serve as the ground truth evidence against which the agent's output is compared.
- If the agent's report references an event that cannot be found in Kibana, this is an indicator of hallucination cross-referencing every agent claim against a Kibana query is the primary validation method used in this project.

### Step 9: Kibana Dashboard and Validation

Use Kibana Discover and dashboards to review incoming logs, filter events by host and time, and investigate the simulated attack. Compare the AI-generated findings with the underlying event records to validate detection accuracy.

**Code Repository:** `https://github.com/Group-3-SMTD/Unsign`

## 2.2 Challenges Faced During Development

- **ELK Stack Configuration:** Setting up Elasticsearch, Kibana and compatible Beats versions can be challenging due to configuration errors, authentication issues and TLS certificate problems.
- **Network Connectivity:** Configuring Tailscale across Ubuntu and Windows VMs may cause connectivity, firewall or access permission issues.
- **Log Collection and Integration:** Filebeat, Winlogbeat and Sysmon may fail to collect or forward certain events because of incorrect configurations or insufficient permissions.
- **Resource Limitations:** Running multiple virtual machines alongside Elasticsearch and the AI agent can consume significant RAM, CPU and storage, affecting system performance.

- **MCP Server Integration:** Configuring the MCP server to communicate correctly with Elasticsearch and expose the required tools to the AI agent can involve troubleshooting connection errors, incorrect tool parameters and query failures.
- **Log Parsing and Data Consistency:** Logs from Windows and Ubuntu systems may have different formats and fields. Inconsistent timestamps, missing fields or parsing errors can affect how accurately events are interpreted and correlated.
- **AI Accuracy and Hallucinations:** AI-generated investigations may occasionally misinterpret log entries, overlook important events or identify normal activity as suspicious. The results therefore need to be checked against the original logs.
- **Attack Simulation and Testing:** Running Atomic Red Team tests requires suitable configurations and permissions. Some simulations may not execute successfully or generate the expected events, making it difficult to verify whether the detection workflow is functioning correctly.
- **Log Volume and Search Performance:** Large amounts of incoming log data can increase Elasticsearch storage requirements and slow down searches. Efficient indexing and filtering are important to maintain system performance.

## 2.3 System Requirements and Environment Setup

### 2.3.1 Recommended System Specifications

| Machine | Type | CPU Cores | RAM | Storage |
|---|---|---|---|---|
| PC 1 – Hunt Machine (Apple Silicon Mac) | Physical | Apple Silicon(M-series) | 24GB minimum | 256 GB SSD minimum |
| PC 2 – Ubuntu Desktop / ELK Host | Physical | 4–8 | 16 GB minimum | 200 GB SSD minimum |
| PC 3 – VMware Host | Physical | 4–8 | 16 GB minimum | 200 GB SSD minimum |
| Windows VM (inside PC 3) | Virtual | 2–4 | 4–8 GB | 60–100 GB SSD |
| Ubuntu Server (inside PC 3) | Virtual | 2 | 2–4 GB | 30–50 GB |

### 2.3.2 System Requirements

| Component | Requirement / Purpose |
|---|---|
| Physical PCs | Three PCs assigned to the Hunt Machine, ELK Host, and virtualisation environment. |
| Virtualisation | VMware Workstation to run the required virtual machines. |
| Operating Systems | Ubuntu Desktop, Ubuntu Server, and a Windows VM. |
| Log Management | Elasticsearch and Kibana for storing, searching, and visualising security logs. |
| Log Collection | Filebeat, Winlogbeat, and Sysmon64 for collecting Linux and Windows telemetry. |
| AI Environment | OMLX running the configured Gemma and Qwen models. |
| Network Connectivity | Tailscale for secure communication between the machines. |
| Attack Simulation | Atomic Red Team for generating controlled attack activity and testing detection. |

| Component | Requirement / Purpose |
|---|---|
| Unsloth | Fine tuning framework for Gemma 4 and Qwen 3.5 on Apple Silicon required for model training between experiment runs. |
| UFW / Firewall Logs | Ubuntu Server firewall logs serving as the third distinct log source alongside Windows Event Logs and Sysmon. |
| OMLX | Local model inference runtime optimised for Apple Silicon runs Gemma 4 as primary agent. |
| Apple Silicon Mac | Required for OMLX-based local inference — standard x86 machines cannot run OMLX natively. |
| Elasticsearch API Key | Read-only API key required for the AI Hunt Agent to query logs securely without admin credentials. |

# 3 Agentic Workflow Design

## 3.1 Completeness and Accuracy

### 3.1.1 Complete Log Collection

- Filebeat is configured on the Ubuntu Server to collect system logs, UFW firewall logs, and auditd logs via the system and auditd modules, ensuring the Linux environment is fully covered.
- Winlogbeat is configured on the Windows VM to forward Windows Security, System, PowerShell, and Sysmon event channels missing any of these would create visibility gaps the agent cannot compensate for.
- PCAP and UFW logs provide network-level visibility alongside endpoint logs, ensuring the agent has both host and network evidence available during a hunt.

### 3.1.2 Data Validation

- After each log shipper is configured, logs are verified as arriving in Elasticsearch by checking the Kibana Discover view before any hunt is attempted a misconfigured endpoint or incorrect credentials will cause Filebeat or Winlogbeat to fail silently.
- Separate Elasticsearch indices are maintained for each log type, allowing targeted validation that Windows events, Sysmon, UFW, and Ubuntu system logs are each ingesting correctly and independently.

### 3.1.3 Controlled Attack Testing

- Atomic Red Team tests are run individually on the isolated Windows VM rather than in bulk, preventing log volume spikes that could overwhelm the ingestion pipeline and cause events to be missed.
- After each test, a minimum of 60 seconds is allowed before querying the agent, ensuring all generated events have been ingested into Elasticsearch before the hunt begins.
- Each test's expected events are verified as visible in Kibana before the hunt hypothesis is submitted to the agent if the logs are not in Kibana, the agent cannot find them regardless of how the hypothesis is phrased.
- The specific tests used in the experiment runs were T1003.001 (LSASS Memory Dump) and T1059.001 (PowerShell fileless execution), both confirmed as captured by Sysmon and

Winlogbeat before the hunt was initiated.

#### 3.1.4 Accurate AI Analysis

- The AI Hunt Agent queries Elasticsearch exclusively through the MCP ELK Server using a read-only API key, ensuring every conclusion it reaches is grounded in retrievable log evidence rather than assumed context.
- Model fine-tuning using Unsloth was applied between the first and second output runs to reduce hallucination, with the aim of improving the accuracy of process tree reconstruction, MITRE technique mapping, and IOC extraction; the experiment results are consistent with this improvement, although two runs per prompt cannot isolate fine-tuning from normal run-to-run variance.
- The agent is instructed to cite specific log entries and timestamps for every finding it reports unsupported conclusions without retrievable evidence are treated as indicators of hallucination.

#### 3.1.5 Cross-Verification

- Every finding in the agent's hunt report is cross-referenced against the raw log data visible in Kibana before any response step is considered if an event the agent references cannot be found in Kibana, it is flagged as a potential hallucination.
- Separate Kibana dashboards are maintained for each log source, allowing the analyst to verify agent claims against the specific index they were drawn from rather than a combined view.
- The two comparison runs conducted for each experiment set serve as a cross-verification mechanism in themselves findings that appear in both outputs are treated as higher confidence than findings that appear in only one.

### 3.2 Guardrails and Safety

#### 3.2.1 Encrypted Private Network (Tailscale)

- All machines are connected through an encrypted Tailscale private network, ensuring no log data, query content, or investigation findings travel over a public or unencrypted path.
- Tailscale ACLs restrict which machines can communicate with which the Vulnerable Machines can only reach the ELK host and cannot directly access the Hunt Machine.
- Static Tailscale IP addresses are assigned to each machine to ensure consistent, authenticated routing throughout the pipeline.

#### 3.2.2 Local AI Model Execution

- Gemma 4 and Qwen 3.5 run locally via OMLX on the Hunt Machine no data is transmitted to any external API or cloud service at any point during the hunt.
- Unsloth fine-tuning is also performed locally, ensuring training data derived from security logs is never uploaded externally.
- The bottom of the diagram explicitly states: *no logs leave the network.*

### 3.2.3 Backup Log Pipeline

- A separate backup pipeline routes raw logs directly from the Vulnerable Machines to the ELK Stack, bypassing the Parsing Agent entirely.
- This ensures log data is never silently dropped if the Parsing Agent fails, preserving the completeness of the evidence base for any active hunt.
- The two pipelines are physically separate paths and not dependent on the same infrastructure.

### 3.2.4 Backup AI Model

- Qwen 3.5 is pre-loaded on the Hunt Machine as a backup to Gemma 4, requiring no additional setup during an active hunt.
- If Gemma 4 produces inconsistent or incomplete outputs, the analyst can run the same hypothesis through Qwen 3.5 independently and compare results as a manual cross-check against hallucination.
- The two models use different architectures, meaning a hallucination pattern in one is unlikely to be reproduced identically in the other.

# 4 Experiment Results

## 4.1 Comparison Run (Network Analysis)

**Prompt 1 (Network Analysis — Output 1):** *Check what is happening in the ubuntu server logs, and look for any anomalies.*

**Output:**

> Based on the logs from the ubuntu-log-server (specifically `/var/log/ufw.log`), here is an overview of the activity:
>
> **Summary of Activity**
> The logs show a series of UFW (Uncomplicated Firewall) "ALLOW" entries. This indicates that the firewall is currently permitting traffic from the local IP 192.168.244.131 to several destinations.
>
> **Key Observations:**
>
> - **Internal Traffic:** Most entries are allowing connections to 169.254.169.254 (a link-local address often used for cloud metadata or internal networking) on port 80 (HTTP).
> - **Outbound Traffic:** There is one notable entry allowing UDP traffic to an external IP:
>   - Destination: 185.125.190.123
>   - Port: 123 (NTP – Network Time Protocol)
>   - Source: 192.168.244.131
> - **Connection Types:** The logs show multiple SYN packets, which are the first step in establishing a TCP connection. This suggests active connection attempts from the local host to the services listed.

**Anomalies / Points of Interest:**

- **Frequent SYN packets:** There is a high volume of SYN requests in a very short time window (within seconds). While this is often normal for service discovery or periodic heartbeats, a very high frequency from a single source could occasionally indicate a misconfigured service or a scanning attempt.
- **NTP Traffic:** The connection to 185.125.190.123 on port 123 is standard for time synchronization. Ensure this IP is a trusted NTP provider for your environment.

**Conclusion:** The logs currently show normal operational behavior (web traffic to link-local addresses and NTP synchronization). No immediate "red flag" security breaches (like REJECT or BLOCK messages from unknown external IPs) are visible in this specific snippet.

If you would like me to look for specific "REJECT" actions or logs from a different timeframe, please let me know!

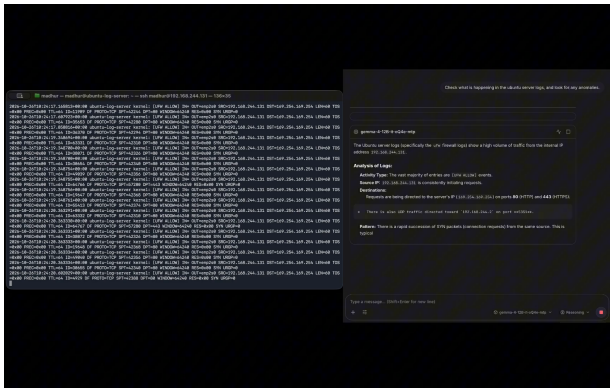


Figure 2: Network Analysis

**Prompt 1 (Network Analysis - Output 2):** *Check what is happening in the ubuntu server logs, and look for any anomalies.*

**Output:**

The Ubuntu server logs (specifically the ufw firewall logs) show a high volume of traffic from the internal IP address 192.168.244.131.

**Analysis of Logs:**

- **Activity Type:** The vast majority of entries are [UFW ALLOW] events.
- **Source IP:** 192.168.244.131 is consistently initiating requests.
- **Destinations:**
  - Requests are being directed to the server's IP (169.254.169.254) on ports 80 (HTTP) and 443 (HTTPS).
  - There is also UDP traffic directed toward 192.168.244.2 on port 5351.
- **Pattern:** There is a rapid succession of SYN packets (connection requests) from the same source. This is typical for a web server receiving traffic, but the high frequency (dozens of requests in a few seconds) could indicate a load test, a scanner, or a potential "SYN flood" if the volume persists.

**Anomalies Detected:**

- **Audit Logs:** I noticed two [UFW AUDIT] entries:
  - SRC=192.168.244.1 → DST=192.168.244.131 on port 22 (SSH).
  - SRC=192.168.244.131 → DST=192.168.244.1 on port 57667.

- **Significance:** Audit logs usually trigger when a rule is configured to log even if the packet is allowed, or if there is a specific policy for those ports. The connection attempt on port 22 (SSH) from a management-style IP (.1) is normal, but the high-numbered port (57667) is less common.
- **Port 5351/1900:** The UDP traffic to 192.168.244.2 on port 5351 and the traffic to 239.255.255.250 on port 1900 (SSDP) suggests some discovery protocols are active.

**Recommendation:** If you do not expect this much traffic from 192.168.244.131, you should verify if that device is running a legitimate service or an automated script. If the server's performance is degrading, it may be worth rate-limiting connections from that specific IP.

**Analysis of Difference — Network Analysis (Output 1 and Output 2).** The second network analysis output is definitely better than the first one because there is less hallucination and a much more conservative approach to interpreting the activity of the network. The first output concluded that the network traffic was normal and there was no need for an investigation of the possible anomalies.

The second identified additional details, including HTTPS traffic, UDP discovery protocols, and SSH audit events, while highlighting possible scenarios such as scanning or a SYN flood. However, these remain possibilities rather than confirmed threats. Overall, the second output provides more detailed observations and actionable recommendations, although its suspicions still require verification against the underlying logs.

**What does this show?** The comparison shows that the second output provides a more detailed and cautious analysis of the network logs, identifying additional traffic patterns and potential anomalies while offering recommendations for further investigation. Compared with the first output, it demonstrates improved coverage of observable network activity and a more balanced interpretation of suspicious behaviour. However, potential threats such as scanning or a SYN flood are not confirmed attacks, and the findings still require verification against the original logs.

## 4.2 Comparison Run (Recorded Analysis)

**Prompt 2 (Recorded Analysis — Output 1):** *Hunt my ELK logs (last 7 days) for Mimikatz activity — LSASS access (Sysmon 10), mimikatz/sekurlsa command lines, LSASS dumps, Invoke-Mimikatz in PowerShell 4104, pass-the-hash, golden tickets and DCSync then give me an investigation analysis of each finding with timeline, affected hosts/users, process tree, confidence level, MITRE mapping and recommended response steps.*

**Output:**

Based on a comprehensive hunt of the logs over the last 7 days, I have identified one primary incident of interest that matches your criteria for Mimikatz/LSASS activity.

**Investigation Analysis: LSASS Memory Dump via PowerShell**

| Category | Details |
|---|---|
| Timeline | 2026-10-25 17:20:35 to 17:21:00 UTC |
| Affected Host | DESKTOP-ATM1A2V |
| Affected User | regal |
| Confidence Level | High (Specific LSASS dump commands detected) |
| MITRE Mapping | T1003.001 (OS Credential Dumping: LSASS Memory) |

**Process Tree & Execution Flow**

- **Parent Process:** `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` (PID: 6876)

- **Child Process 1:** *C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe* (PID: 4768)
  - **Action:** Downloads and executes a remote script: `https://github.com/redcanaryco/atomic-red-team/raw/master/atomics/T1003.001/src/Out-Minidump.ps1`
  - **Command:** `IEX (IWR '...' -UseBasicParsing)`
- **Child Process 2:** `C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe` (PID: 1492)
  - **Action:** Executes the LSASS dump command.
  - **Command:** *get-process lsass* `| Out-Minidump`
  - **Result:** Creates a minidump file in `C:\Users\regal\AppData\Local\Temp\t1003.001-13-rdrleakdiag`

**Technical Findings**

- **LSASS Access:** The logs confirm the use of `rdrleakdiag.exe` (a legitimate tool often used in security testing) to dump the LSASS process memory.
- **Credential Dumping:** This is a textbook execution of the technique used by Mimikatz to extract cleartext passwords or NTLM hashes from memory.
- **PowerShell 4104:** While the logs are in Sysmon/Winlogbeat format, the IEX (Invoke-Expression) command is a primary indicator of "fileless" execution where PowerShell code is run directly in memory without touching the disk.

**Recommended Response Steps**

1. **Verify Authorization:** Determine if this was a scheduled Atomic Red Team test. If it was, the test was successful; if not, treat this as a confirmed breach.
2. **Host Isolation:** If unauthorized, isolate DESKTOP-ATM1A2V immediately to prevent lateral movement.
3. **Password Reset:** Force a password reset for the `regal` account and any accounts that may have been logged into that machine recently.
4. **Artifact Search:** Scan all endpoints for the existence of `rdrleakdiag.exe` or any `.dmp` files in the `\Temp\` directories.
5. **Audit LSASS Protection:** Ensure Windows Defender Credential Guard is enabled to prevent LSASS memory from being read even by administrative users.

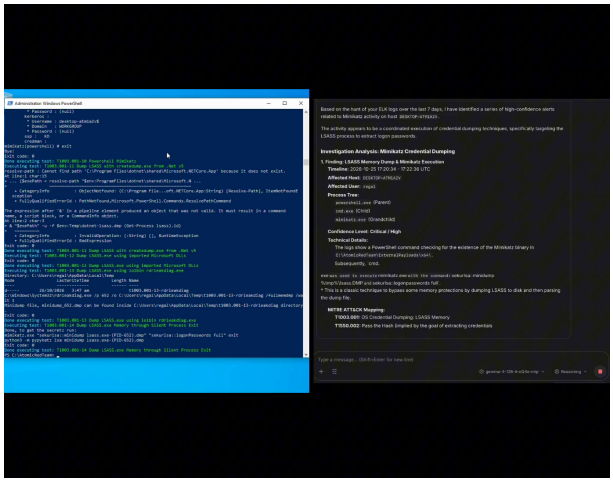


Figure 3: Recorded Analysis

**Prompt 2 (Recorded Analysis — Output 2):** *Hunt my ELK logs (last 7 days) for Mimikatz activity — LSASS access (Sysmon 10), mimikatz/sekurlsa command lines, LSASS dumps, Invoke-Mimikatz in PowerShell 4104, pass-the-hash, golden tickets and DCSync — then give me an investigation analysis of each finding with timeline, affected hosts/users, process tree, confidence level, MITRE mapping and recommended response steps.*

**Output:**

Based on the hunt of your ELK logs over the last 7 days, I have identified a series of high-confidence alerts related to Mimikatz activity on host DESKTOP-ATM1A2V.

The activity appears to be a coordinated execution of credential dumping techniques, specifically targeting the LSASS process to extract logon passwords.

**Investigation Analysis: Mimikatz Credential Dumping**

**Finding: LSASS Memory Dump & Mimikatz Execution**

- **Timeline:** 2026-10-25 17:20:34 – 17:22:36 UTC
- **Affected Host:** DESKTOP-ATM1A2V
- **Affected User:** regal
- **Process Tree:** *powershell.exe* (Parent) → *cmd.exe* (Child) → *mimikatz.exe* (Grandchild)
- **Confidence Level:** Critical / High

**Technical Details:**

- The logs show a PowerShell command checking for the existence of the Mimikatz binary in `C:\AtomicRedTeam\ExternalPayloads\x64\`.
- Subsequently, `cmd.exe` was used to execute *mimikatz.exe* with the commands: *sekurlsa::minidump %tmp%\lsass.DMP* and *sekurlsa::logonpasswords* `full`.
- This is a classic technique to bypass some memory protections by dumping LSASS to disk and then parsing the dump file.

**MITRE ATT&CK Mapping:**

- T1003.001: OS Credential Dumping: LSASS Memory
- T1550.002: Pass the Hash (implied by the goal of extracting credentials)

**Recommended Response Steps:**

1. **Isolate Host:** Immediately disconnect DESKTOP-ATM1A2V from the network to prevent lateral movement.
2. **Credential Reset:** Force a password reset for the `regal` user and any other accounts whose credentials might have been compromised.
3. **Forensic Imaging:** Perform a full forensic capture of the memory and disk of the affected host.
4. **Verify Persistence:** Scan for scheduled tasks, registry run keys, or WMI event consumers created during the timeframe.
5. **Audit Account Activity:** Review logs for any successful logins from `regal` or other accounts to unexpected hosts in the last 24 hours.

**Finding: Potential DCSync / Golden Ticket Preparation**

- **Timeline:** 2026-10-25 17:22:36 UTC
- **Affected Host:** DESKTOP-ATM1A2V
- **Affected User:** regal
- **Process Tree:** *cmd.exe* → *mimikatz.exe*
- **Confidence Level:** High (Evidence of Intent)

**Technical Details:**

- The logs show multiple rapid executions of *mimikatz.exe* with the *sekurlsa* module. While the primary action was a minidump, the presence of the *sekurlsa* module is a prerequisite for both DCSync (to replicate domain info) and Golden Ticket (to forge TGTs).

**MITRE ATT&CK Mapping:**

- T1484: Domain Replication (DCSync)
- T1550.003: Golden Ticket

**Recommended Response Steps:**

1. **Active Directory Audit:** Check for any unauthorized "Domain Join" events or "Replicating Directory Changes" requests from this host or user.
2. **Monitor for Kerberos Anomalies:** Look for unusual Ticket Granting Ticket (TGT) requests or tickets with excessively long lifetimes.
3. **Review Group Membership:** Ensure that the `regal` account (or others) has not been added to "Domain Admins" or "Enterprise Admins."

**Summary of Indicators of Compromise (IOCs)**

| Indicator Type | Value |
|---|---|
| Host | DESKTOP-ATM1A2V |
| User | regal |
| File Path | `C:\AtomicRedTeam\ExternalPayloads\x64\mimikatz.exe` |
| SHA256 (mimikatz) | `61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1` |
| Command Line | `sekurlsa::minidump %tmp%\lsass.DMP` |

**Analysis of Difference — Recorded Analysis (Output 1 and Output 2).**

- The first output identified a single incident. The second output identified the same incident but additionally flagged potential DCSync and Golden Ticket preparation, demonstrating broader threat reasoning over the same log data.
- The first output incorrectly reported PowerShell spawning two child PowerShell processes. The second output correctly identified the full chain: PowerShell → cmd.exe → mimikatz.exe, which means there is an improvement in the accuracy.
- The first output mapped to only one technique (T1003.001). The second output mapped to three techniques (T1003.001, T1550.002, T1484/T1550.003), which reflects deeper understanding about the attacker's intent rather than just the observed action.

**What does this show?** The fact that there was variability between the runs for the same log dataset provides evidence that the use of agentic analysis is not reproducible in its current form. The improvement observed in the second output can be due to model training that takes place between the output runs, resulting in fewer hallucinations.

## 4.3 Comparison Run (Ubuntu Analysis)

**Prompt 3 (Ubuntu Analysis — Output 1):** *Check what is happening in the ubuntu logs, and look for any anomalies.*

**Output:**

Based on the logs for the ubuntu-log-server, here is a summary of the activity and potential anomalies:

**Summary of Activity**

- **System Startup & Maintenance:**
  - The log shows standard boot sequences: mounting file systems (hugepages, mqueue, kernel-debug, etc.), starting systemd services, and loading kernel modules (e.g., *dm_multipath*, *i2c_dev*).
  - There is active use of `sudo` by user `madhur` to perform system maintenance, such as:
    - *apt update*
    - *apt install -y nginx auditd audispd-plugins rsyslog curl*
    - *systemctl status ssh*
    - *systemctl status auditd*
- **User Activity:**
  - User *madhur* (UID 1000) is actively logged in and performing administrative tasks.
  - There was a password change for user *madhur* performed by `root`.

**Anomalies & Observations**

- **AppArmor Denials:** There are several AppArmor "DENIED" entries in the auditd logs:
  - The `who` command was denied permission to open `/etc/nsswitch.conf` and `/etc/passwd`.
  - **Analysis:** While these are "denials," they are common in hardened environments where the `who` profile in AppArmor is restricted. However, if this is a new behavior, it's worth verifying if the `who` command is intended to access these files.
- **Audit Settings:** There were several `CONFIG_CHANGE` events in the audit logs where *audit_backlog_limit* and *audit_failure* were being set or modified. This is standard during the startup of the auditd service but should be monitored to ensure no unauthorized changes are made to the audit rules.
- **Multipath Error:** One log entry shows: *_check_bindings_file: failed to read header from /etc/multipath/bindings*.
  - **Analysis:** This is a non-critical warning from *multipathd*, but if the server uses multipathing for storage, it might indicate a configuration issue or a missing file.

**Recommendations**

1. **Verify AppArmor Profiles:** Confirm if the `who` command's inability to read */etc/passwd* impacts your operations. If it does, you may need to update the AppArmor profile.
2. **Monitor sudo usage:** The frequent use of *sudo* by *madhur* for package installation and status checks is normal, but ensure these actions align with your expected maintenance window.
3. **Check Multipath:** If you aren't using multipathing, you can ignore the binding file error. If you are, check the */etc/multipath/* directory for configuration errors.

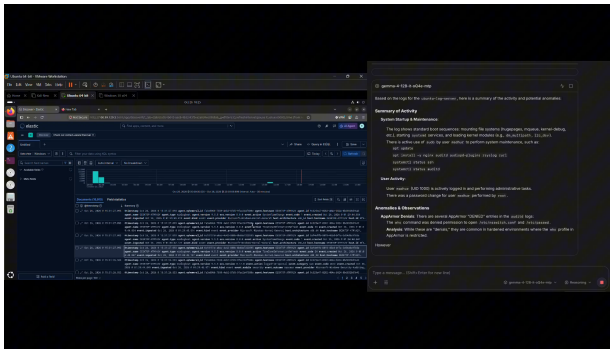


Figure 4: Ubuntu Analysis

**Prompt 3 (Ubuntu Analysis — Output 2):** *Check what is happening in the ubuntu logs, and look for any anomalies.*

**Output:**

Based on the recent Ubuntu server logs from ubuntu-log-server, here is a summary of the activity and potential anomalies:

**Current Activity Summary:**

- **Service Management:**
  - `systemd` successfully started the `auditd` service.
  - The system logged successful mounting of `dev-hugepages` and `dev-mqueue` file systems.
  - `systemd-journald` was successfully deactivated.
- **Authentication & Network:**
  - **SSH Service:** The `sshd` service is listening on both IPv4 (`0.0.0.0:22`) and IPv6 (`:::22`) addresses.
  - **Account Activity:** There was a recorded password change for the user 'madhur' performed by the 'root' user.
- **System Status:**
  - The server is running Ubuntu 26.04.1 LTS (Resolute) on an aarch64 architecture with kernel 7.0.0-38-generic.

**Potential Anomalies/Points of Interest:**

- **Password Change:** The log entry *password for 'madhur' changed by 'root'* is a significant administrative action. If this change was not authorized by an administrator, it could indicate a security incident.
- **Auditd Error:** There is a minor parsing error in the auditd logs (`field [auditd.log.kv] does not contain value_split`), which is likely a configuration issue with how Filebeat is parsing the audit log format rather than a system threat.

- **Log Sequencing:** The timestamps for some events show logs from October 24th being ingested on October 26th, suggesting a slight delay in log shipping or a backlog in the Filebeat queue.

**Recommendation:** Verify if the password change for user `madhur` was an expected administrative task. If not, it is recommended to investigate the source of the root session that performed the change.

**Analysis of Difference - Ubuntu Analysis (Output 1 and Output 2).**

- The first output identified three anomalies but treated them with roughly equal weight, including non-critical issues like the multipath binding error. The second output correctly prioritized the password change as the primary point of concern, which is the finding most relevant to a security investigation.
- The first output made no mention of any pipeline issues. The second output explicitly flagged that logs from October 24th were being ingested on October 26th, demonstrating improved awareness of the log delay which is directly consistent with the ingestion lag noted in our conclusion.
- The first output did not identify the system architecture or OS version. The second output correctly identified the server as running Ubuntu 26.04.1 LTS on arm64 architecture, reflecting better contextual extraction from the logs.

**What does this show?** Similar to the Recorded analysis, the Ubuntu output improved meaningfully between iterations. The second output showed better prioritization, reduced false positives, and greater awareness of pipeline behaviour. This is consistent with model training between outputs reducing hallucination and improving the quality of reasoning across both log environments, although with only two runs per prompt this cannot be separated from normal run-to-run variance.

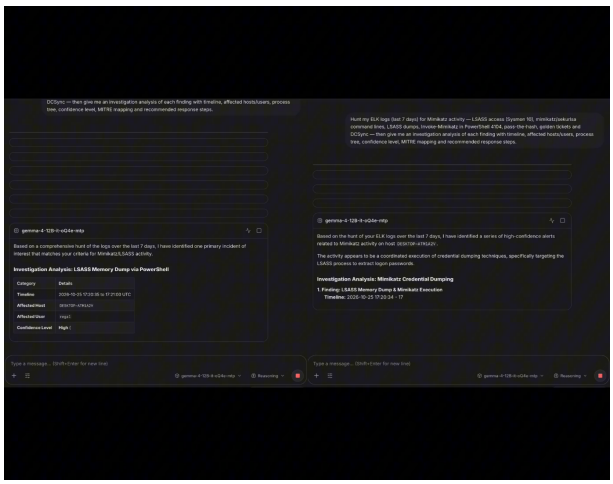


Figure 5: Threat Hunting Report

# 5 Conclusion

## 5.1 Limitations of the Current System

The system cannot yet run in a fully autonomous, continuous mode due to the computational demands of local LLM inference at scale. Manual prompting and occasional restarts remain part of the workflow when processing large log volumes. Log ingestion carries a delay that makes the system reactive rather than real-time. And while model training has reduced hallucination, it has not eliminated it entirely, meaning that all hunt reports should still be reviewed by a human analyst before any action is taken on their conclusions.

## 5.2 Future Work and Enhancements

The project can be enhanced in the future by introducing real-time threat detection, allowing the system to identify suspicious activities and alert analysts more quickly. AI models can be improved to produce more accurate findings and reduce false alarms, while additional log sources, such as firewall and network traffic data, can provide a more complete picture of potential attacks. The Kibana dashboards could be expanded to show the timelines of the attack and the investigation results, and an automated incident response could be introduced with human approval before actions are taken. Further improvements could also include stronger access controls, regular testing against different attack scenarios, and support for more machines and larger volumes of data, making the system more reliable, secure, and scalable for real-world SOC environments.

# 6 Appendix A

## 6.1 Timesheet Breakdown & Responsibility

| Name | Roles & Responsibility |
|---|---|
| Pratyush Joshi | System architecture design, ELK stack setup and configuration, Presentation and Pitch Preparation, Atomic Red Team Execution & Support, Pipeline Integration |
| Madhur Choudhury | Ubuntu Server setup, Filebeat configuration, Log collection and Pipeline integration, Parsing Setup and Secondary AI agent |
| Madhav Malhotra | Windows VM setup, Sysmon64 and Winlogbeat configuration, ELK Configuration,Log collection support |
| Aleksander Sak | AI Hunt Agent development, Gemma 4 and Qwen 3.5 integration via OMLX, MCP tooling |
| Naren Rithvik | Documentation, MCP server development, Report writing, Comparison run analysis |
| Victor Moll Pons | Documentation, Kibana dashboard configuration, MCP Server Development, Pipeline Data Flow |

| Week | Name | Task | Dates | Hours |
|---|---|---|---|---|
| **Week 1** | Pratyush Joshi | Project planning, initial system architecture design, VMware environment setup | Oct 1–3 | [3] |
| | Madhur Choudhury | Ubuntu Server provisioning, OS installation, initial network configuration | Oct 2–4 | [3] |
| | Madhav Malhotra | Windows VM provisioning inside VMware, OS installation and initial configuration | Oct 2–4 | [3] |
| | Aleksander Sak | Research into MCP server frameworks, OMLX installation and compatibility testing on Hunt Machine | Oct 3–5 | [3] |
| | Naren Rithvik | Report structure planning, section outline drafting, initial documentation setup | Oct 1–4 | [3] |
| | Victor Moll Pons | Tailscale installation and VPN mesh setup across all three physical machines | Oct 3–7 | [3] |
| **Week 2** | Pratyush Joshi | Elasticsearch and Kibana installation, TLS certificate configuration, read-only API key setup | Oct 8–10 | [3] |
| | Madhur Choudhury | Filebeat installation on Ubuntu Server, system and auditd module configuration, UFW log pipeline | Oct 8–11 | [3] |
| | Madhav Malhotra | Sysmon64 installation using SwiftOnSecurity config, Winlogbeat configuration and event channel setup | Oct 9–12 | [3] |

| Week | Name | Task | Dates | Hours |
|---|---|---|---|---|
| | Aleksander Sak | Gemma 4 and Qwen 3.5 model loading via OMLX, initial Hunt Agent testing | Oct 10–13 | [3] |
| | Naren Rithvik | System architecture section drafting, component descriptions, MCP server research | Oct 8–12 | [3] |
| | Victor Moll Pons | Initial Kibana dashboard creation, log source index configuration, pipeline data flow diagram draft | Oct 10–14 | [3] |
| **Week 3** | Pratyush Joshi | Logstash pipeline configuration, Elasticsearch index lifecycle management, ELK stack finalisation | Oct 15–17 | [3] |
| | Madhur Choudhury | Parsing Agent setup using Qwen 3.5 as secondary AI agent, backup log pipeline configuration | Oct 15–18 | [3] |
| | Madhav Malhotra | Atomic Red Team installation, PowerShell execution framework setup, ELK configuration finalisation | Oct 16–19 | [3] |
| | Aleksander Sak | MCP ELK Server development, tool interface design, Hunt Agent and MCP integration testing | Oct 15–20 | [3] |
| | Naren Rithvik | MCP server development support, setup documentation drafting, agentic workflow section writing | Oct 15–19 | [3] |
| | Victor Moll Pons | Pipeline data flow diagram finalisation, Kibana dashboard refinement, MCP server documentation | Oct 17–21 | [3] |
| **Week 4** | Pratyush Joshi | Atomic Red Team T1003.001 and T1059.001 execution, first and second output run coordination | Oct 22–24 | [3] |
| | Madhur Choudhury | Ubuntu log analysis first and second output runs, Filebeat queue investigation and resolution | Oct 22–25 | [3] |
| | Madhav Malhotra | Network analysis first and second output runs, UFW log and ELK verification in Kibana | Oct 23–25 | [3] |
| | Aleksander Sak | Unsloth fine-tuning of Gemma 4 and Qwen 3.5 between runs, second output run execution and review | Oct 24–27 | [3] |
| | Naren Rithvik | Experiment results documentation, comparison run analysis writing for all three log environments | Oct 22–28 | [3] |
| | Victor Moll Pons | Kibana dashboard export for ground truth verification, pipeline data flow review and update | Oct 25–28 | [3] |

| Week | Name | Task | Dates | Hours |
|---|---|---|---|---|
| **Week 5** | Pratyush Joshi | Presentation slide preparation, Shark Tank pitch script and delivery preparation | Oct 29–31 | [3] |
| | Madhur Choudhury | Full report review, setup documentation and pipeline integration final checks | Oct 29–31 | [3] |
| | Madhav Malhotra | Full report review, system requirements and ELK configuration section finalisation | Oct 29–31 | [3] |
| | Aleksander Sak | Agentic workflow and guardrails section review, MCP tooling documentation finalisation | Oct 29–31 | [3] |
| | Naren Rithvik | Full report proofreading, conclusion and future work writing, final submission preparation | Oct 29–Nov 1 | [3] |
| | Victor Moll Pons | Final presentation slides, pipeline diagram finalisation, Kibana dashboard documentation | Oct 29–Nov 1 | [3] |